

Simple Storage Service(S3)

1.Create an S3 bucket and upload some objects to S3.

Objective: Create an Amazon S3 bucket and upload files (objects) into the bucket using the AWS Management Console.

Step 1: Log in to AWS Management Console

1. Open your browser.
 2. Sign in to the AWS Management Console.
 3. In the search bar, type **S3**.
 4. Click **Amazon S3**.
-

Step 2: Create an S3 Bucket

1. Click **Create bucket**.
2. Configure the bucket:

Field	Value
AWS Region	US East (N. Virginia) us-east-1 (or your assigned region)
Bucket Name	rajkumari-demo-bucket-2026 (<i>must be globally unique</i>)

Step 3: Configure Bucket Settings

Leave the default settings.

Object Ownership

Select

ACLs disabled (Recommended)

Field

Value

Block Public Access

Keep all four options checked.

✓ Block all public access

This is the recommended and secure configuration.

Bucket Versioning

Choose

Disable

(Unless your assignment specifically asks to enable versioning.)

Default Encryption

Keep

Server-side encryption with Amazon S3 managed keys (SSE-S3)

Click

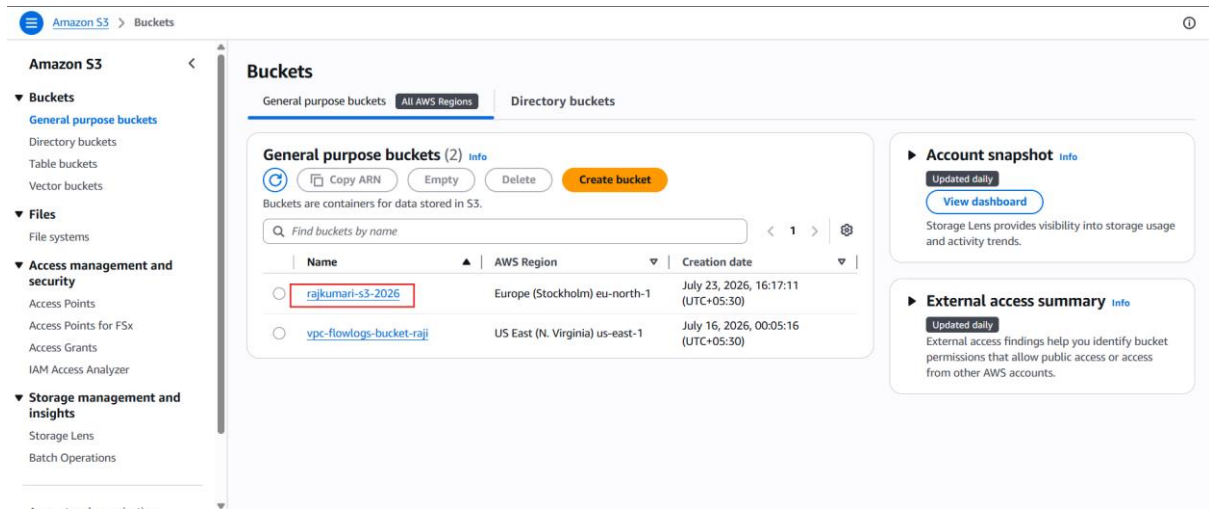
Create bucket

You should see

Successfully created bucket

Field

Value



Step 4: Open the Bucket

Click on the bucket name.

Example

rajkumari-s3-2026

Step 5: Upload Objects

Inside the bucket

Click

Upload

Then click

Add files

Select some files from your computer.

Step 6: Upload

After selecting the files,

Click

Upload

Wait until you see

Upload succeeded

Click

Close

Amazon S3

Upload succeeded

For more information, see the [Files and folders](#) table.

After you navigate away from this page, the following information is no longer available.

Summary

Destination

s3://rajkumari-s3-2026

Succeeded

2 files, 30.0 B (100.00%)

Failed

0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (2 total, 30.0 B)

Find by name

Name	Folder	Type	Size	Status	Error
hello.txt	-	text/plain	15.0 B	Succeeded	-
sample.txt	-	text/plain	15.0 B	Succeeded	-

Step 9: Download an Object (Optional)

Select a file.

Click

Download

Verify that it downloads successfully.

The screenshot displays the AWS S3 console interface. The top navigation bar shows the AWS logo and various icons. The main content area is divided into a left sidebar and a right pane. The sidebar contains a tree view with 'Amazon S3' expanded, showing 'Buckets' and 'Files'. The right pane shows the details for the object 'hello.txt' in the bucket 'rajkumari-s3-2026'. The object's size is 15 B and it is marked as 'Done'. Below the object name, there are buttons for 'Copy S3 URI', 'Download', and 'Open'. A dropdown menu for 'Object actions' is also visible. The 'Properties' tab is selected, showing the 'Object overview' section with the owner's ID: 8428a3e13f1864f9f3247847c2bc79dc9991e91c3322156fb. Below the console, a Windows File Explorer window is open, showing the 'hello.txt' file in the 'RajKumari - Personal' folder. A red box highlights the file's details in the table below.

Name	Date modified	Type	Size
✓ Today			
hello.txt	23-07-2026 16:26	Text Document	1 KB
✓ Yesterday			

Step 10: Delete an Object (Optional)

Select a file.

Click

Delete

Type

permanently delete

Click

Delete objects

☰ Amazon S3

✔ Successfully deleted objects
View details below.

Summary

Source s3://rajkumari-s3-2026	Successfully deleted ✔ 1 object, 15.0 B	Failed to delete 0 objects
----------------------------------	--	-------------------------------

Failed to delete | Configuration

⊕ Failed to delete (0)

🔍 Find objects by name

Name ▲	Folder ▼	Type ▼	Last modified ▼	Size ▼	Error ▼
No objects failed to delete.					

2. Deploy a static website in the S3 bucket.

Objective: Host a static website (HTML, CSS, JavaScript) using an Amazon S3 bucket.

Prerequisites

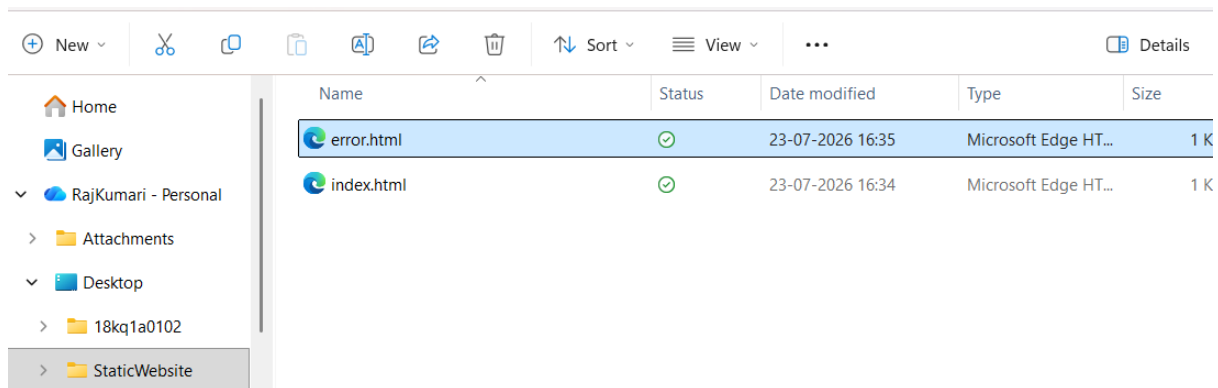
Before starting, ensure you have:

- ✔ An AWS account
- ✔ An existing S3 bucket (or create a new one)
- ✔ A simple website with at least:
 - index.html
 - (Optional) error.html
 - Any CSS, JavaScript, or image files

Step 1: Create a Sample Website

Create a folder named **StaticWebsite** on your computer.

Inside it, create a file named **index.html** with the following content:



Step 2: Open Amazon S3

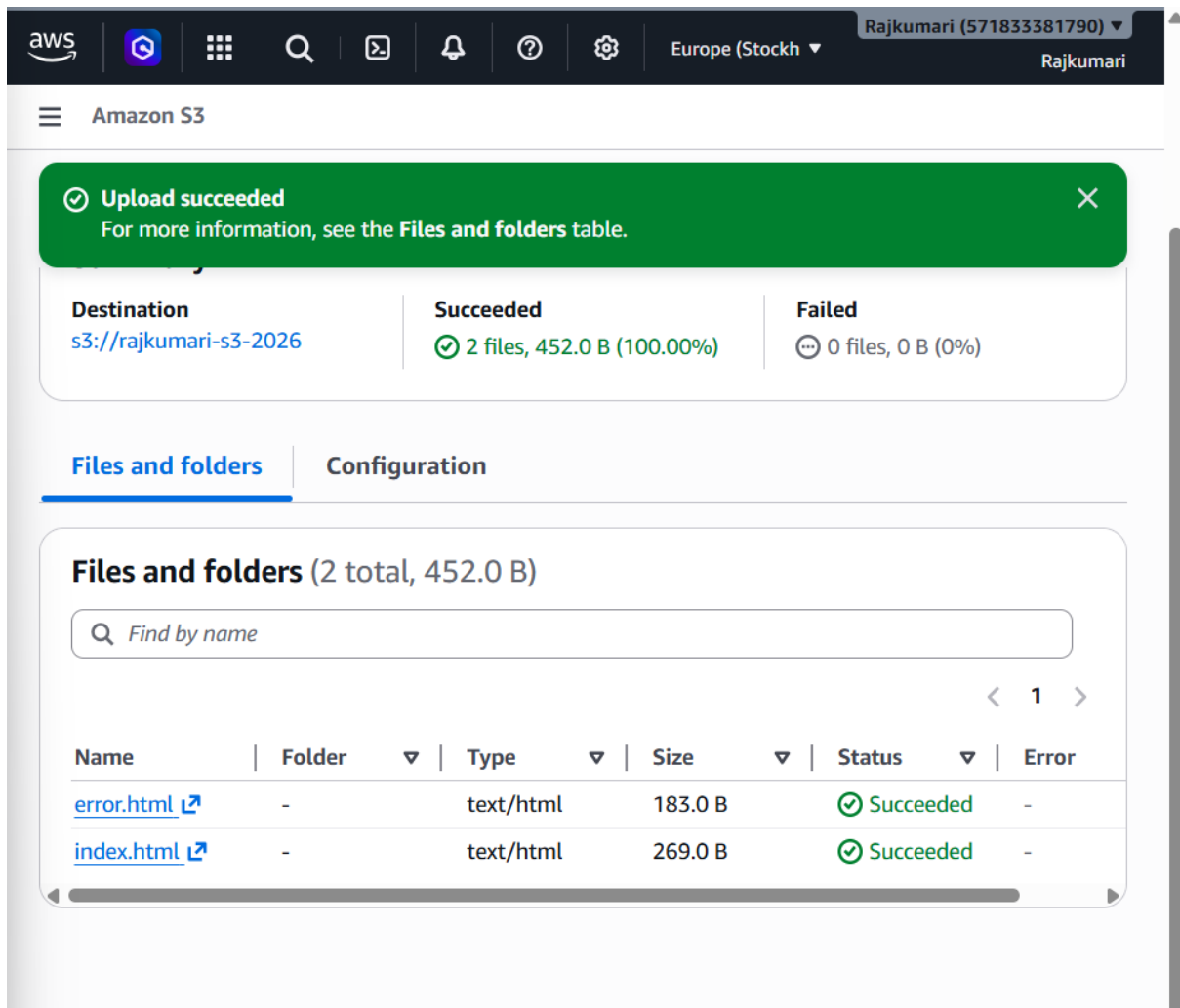
1. Sign in to the AWS Management Console.
2. Search for **S3**.
3. Open **Amazon S3**.
4. Select your bucket.

Example:

Rajkumari-s3-2026

Step 3: Upload Website Files

1. Click **Upload**.
2. Click **Add files**.
3. Select:
 - index.html
 - error.html
 - Any CSS/JS/image files
4. Click **Upload**.
5. Wait until the upload is successful.

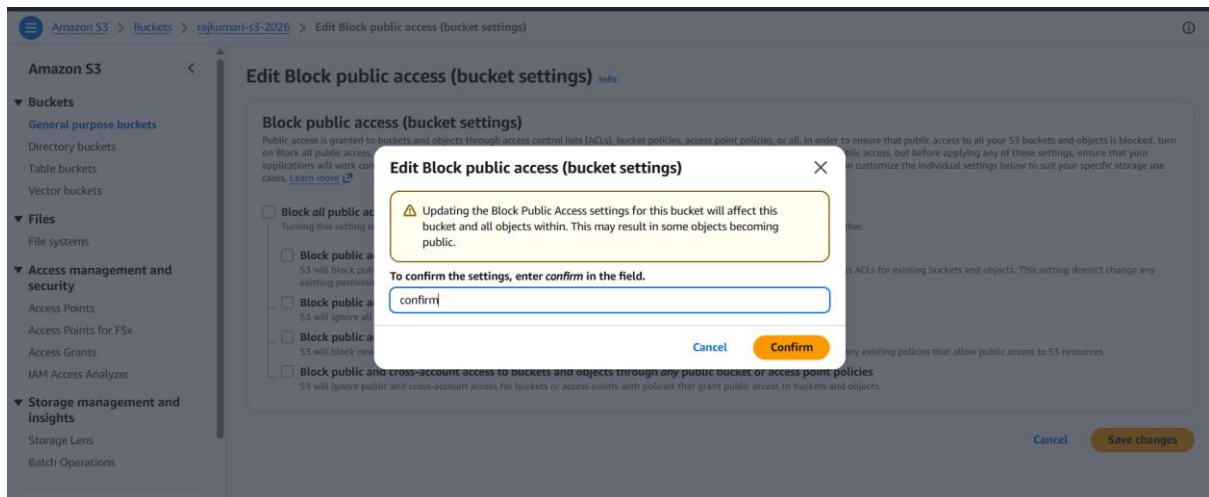


Step 4: Disable Block Public Access

To make the website publicly accessible:

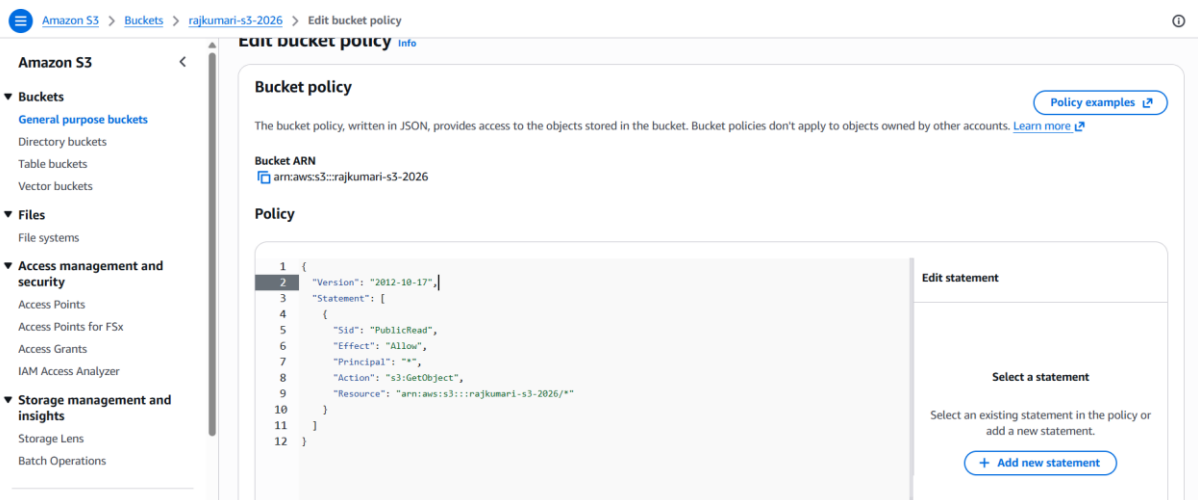
1. Open your bucket.
2. Go to the **Permissions** tab.
3. Under **Block public access (bucket settings)**, click **Edit**.
4. Uncheck **Block all public access**.
5. Confirm the warning by typing **confirm** (if prompted).
6. Click **Save changes**.

Note: This is required for public website hosting. Do this only for buckets intended to host public content.



Step 5: Add a Bucket Policy

1. Stay on the **Permissions** tab.
2. Scroll to **Bucket policy**.
3. Click **Edit**.
4. Replace YOUR-BUCKET-NAME with your actual bucket name in the following policy:



Step 6: Enable Static Website Hosting

1. Open the **Properties** tab.
2. Scroll to **Static website hosting**.
3. Click **Edit**.

4. Choose **Enable**.
5. Under **Hosting type**, select **Host a static website**.
6. Set:
 - **Index document:** index.html
 - **Error document:** error.html
7. Click **Save changes**.

The screenshot shows the AWS Management Console interface for editing static website hosting on a bucket named 'rajkumari-s3-2026'. The left sidebar contains a navigation menu with categories: Amazon S3, Buckets, Files, Access management and security, and Storage management and insights. The main content area is titled 'Edit static website hosting' and includes the following sections:

- Static website hosting:** Radio buttons for 'Disable' and 'Enable'. The 'Enable' option is selected.
- Hosting type:** Radio buttons for 'Host a static website' and 'Redirect requests for an object'. The 'Host a static website' option is selected. A note below states: 'Use the bucket endpoint as the web address. [Learn more](#)'. The 'Redirect requests for an object' option has a note: 'Redirect requests to another bucket or domain. [Learn more](#)'.
- Index document:** A text input field containing 'index.html'. A note above the field says: 'Specify the home or default page of the website.'
- Error document - optional:** A text input field containing 'error.html'. A note above the field says: 'This is returned when an error occurs.'
- Redirection rules - optional:** This section is partially visible at the bottom.

A light blue information box is present, stating: 'For your customers to access content at the website endpoint, you must make all your content publicly readable. To do so, you can edit the S3 Block Public Access settings for the bucket. For more information, see [Using Amazon S3 Block Public Access](#)'.

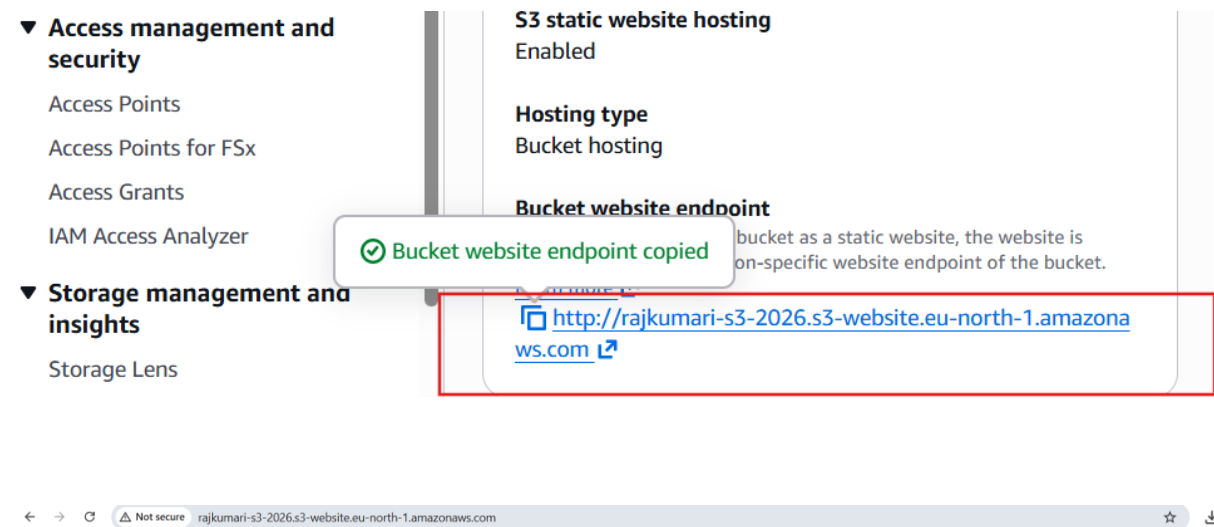
Step 7: Open the Website

After enabling static website hosting, AWS displays a **Bucket website endpoint** similar to:

http://rajkumari-demo-bucket-2026.s3-website-us-east-1.amazonaws.com

Click or copy this URL into your browser.

If everything is configured correctly, you should see:



3. Enable cross-region replication on S3 buckets.

Objective: Configure **Cross-Region Replication (CRR)** to automatically replicate objects from a source S3 bucket in one AWS Region to a destination S3 bucket in another AWS Region.

Note: Cross-Region Replication works **only for new objects uploaded after replication is enabled**. Existing objects are not replicated automatically unless you use **S3 Batch Replication**.

Prerequisites

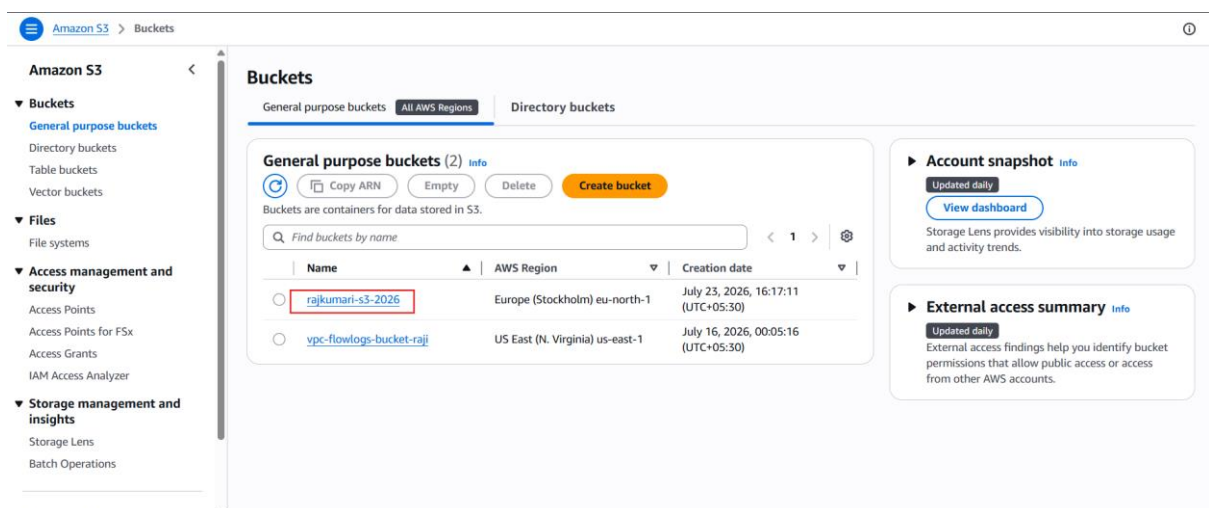
Before configuring CRR, ensure:

- ☒ Two S3 buckets exist.
- ☒ The buckets are in **different AWS Regions**.
- ☒ **Versioning is enabled** on both buckets.

- ☒ You have IAM permissions to manage S3 replication.

Step 1: Create the Source Bucket

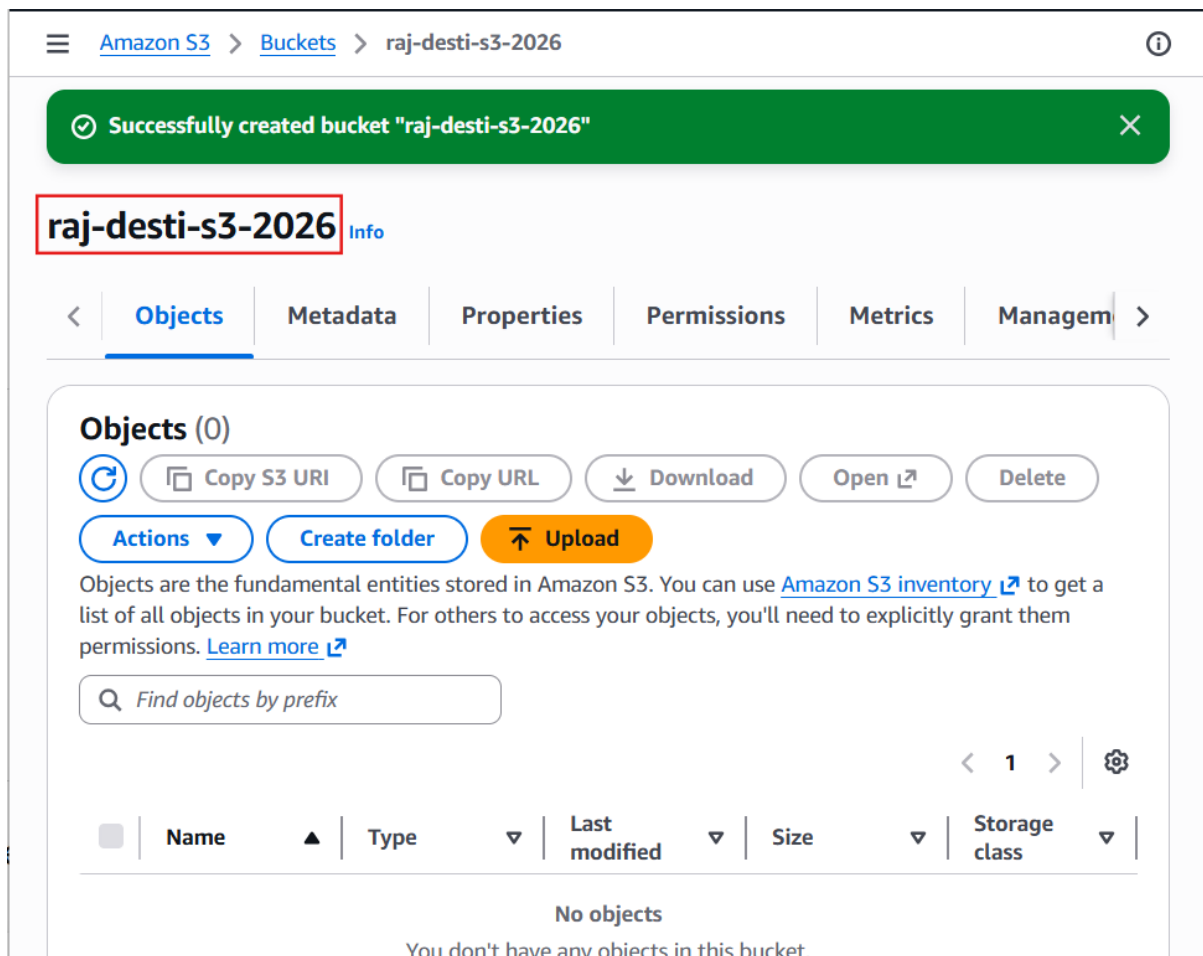
1. Sign in to the AWS Management Console.
2. Open **Amazon S3**.
3. Click **Create bucket**.
4. Configure:
 - **Bucket name:** rajkumari-s3-2026
 - **Region:** Europe(stockholm)eu-north-1
5. Leave other settings as default.
6. Click **Create bucket**



Step 2: Create the Destination Bucket

1. Click **Create bucket** again.
2. Configure:
 - **Bucket name:** raj-desti-s3-2026
 - **Region:** US West (Oregon) - us-west-2 (or any different Region)
3. Leave other settings as default.

4. Click **Create bucket**.



Step 3: Enable Versioning on the Source Bucket

1. Open the **source bucket**.
2. Go to the **Properties** tab.
3. Scroll to **Bucket Versioning**.
4. Click **Edit**.
5. Select **Enable**.
6. Click **Save changes**.

Amazon S3

> Buckets

> rajkumari-s3-2026

Amazon S3

<

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

▼ Files

File systems

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and insights

Storage Lens

Batch Operations

✓ Successfully edited Bucket Versioning

To transition, archive, or delete older object versions, [configure lifecycle rules](#) for this bucket.

✕

Creation date

July 23, 2026, 16:17:11 (UTC+05:30)

Bucket Versioning

Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

Enabled

Multi-factor authentication (MFA) delete

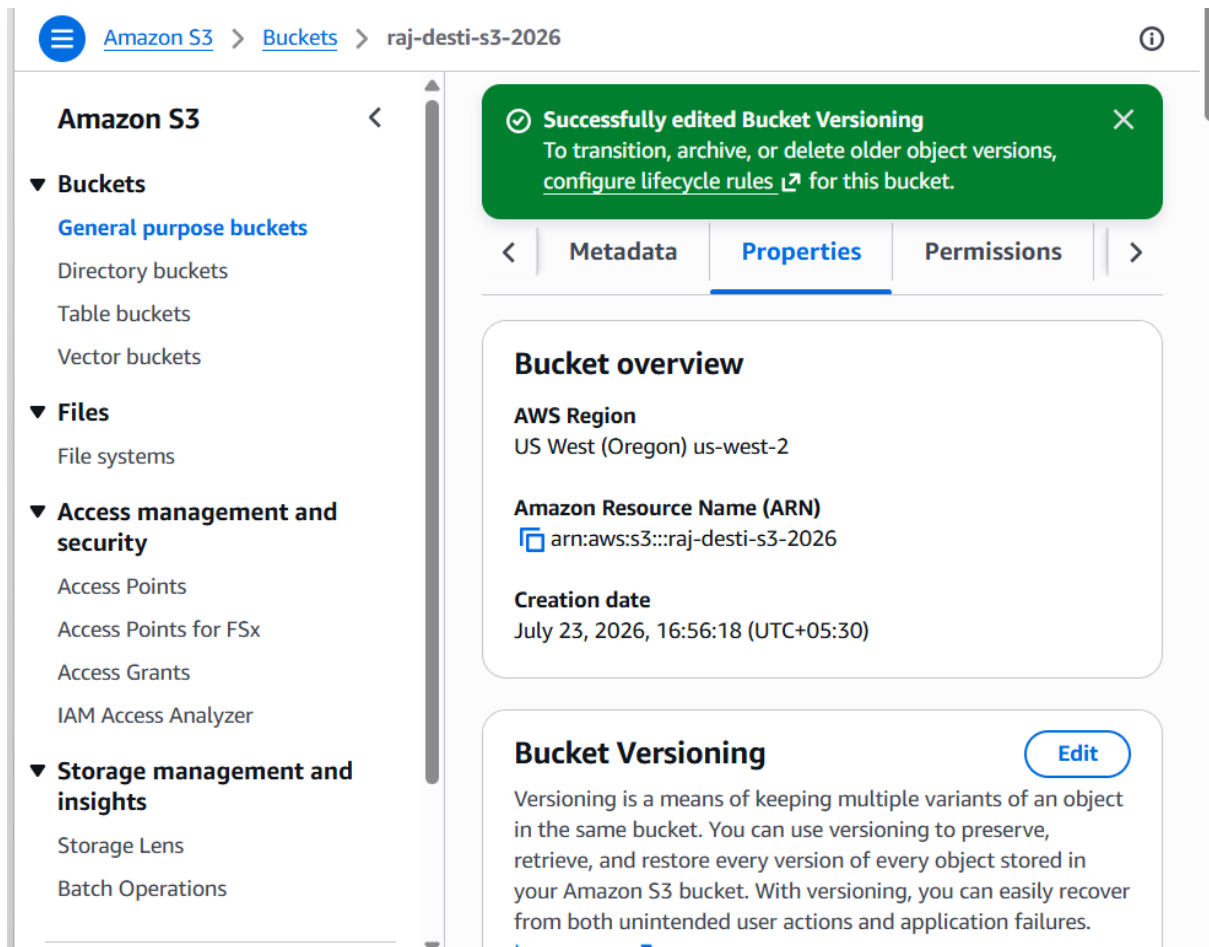
An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. [Learn more](#)

Disabled

Step 4: Enable Versioning on the Destination Bucket

Repeat the same steps for the **destination bucket**.

Important: Replication cannot be configured unless **versioning** is enabled on both buckets.



Step 5: Configure Replication

1. Open the **source bucket**.
2. Go to the **Management** tab.
3. Scroll to **Replication rules**.
4. Click **Create replication rule**.

Step 6: Configure the Replication Rule

Set the following:

- **Rule name:** CrossRegionReplication
- **Status:** Enabled

- **Rule scope:** Apply to all objects in the bucket (or specify a prefix if required)

Click **Next** (if prompted).

Step 7: Select the Destination Bucket

1. Choose **Destination bucket in another Region**.
 2. Select your destination bucket:
 - rajkumari-destination-bucket-2026
 3. Save the configuration.
-

Step 8: Configure the IAM Role

AWS prompts for an IAM role.

Choose:

- **Create new IAM role (recommended)**

AWS automatically creates the required role with the necessary permissions.

Continue to the next step.

Step 9: Review and Save

Review the replication settings.

Click **Save**.

The replication rule status should show **Enabled**.

Amazon S3 > Buckets > rajkumari-s3-2026 > Replication rules

Replication configuration successfully updated

If changes to the configuration aren't displayed, choose the refresh button. Changes apply only to new objects. To replicate existing objects with this configuration, choose **Create replication job**.

Create replication job

Replication rules (1)

View details

Edit rule

Delete

Actions

Create replication rule

Use replication rules to define options you want Amazon S3 to apply during replication such as server-side encryption, replica ownership, transitioning replicas to another storage class, and more. Replication also includes any annotations attached to your objects. [Learn more](#)

< 1 >

Replication rule name	Status	Destination bucket	Destination Region	Priority	Scope
☐ CrossRegionReplication	Enabled	s3://raj-desti-s3-2026	US West (Oregon) us-west-2	0	Entire bucket

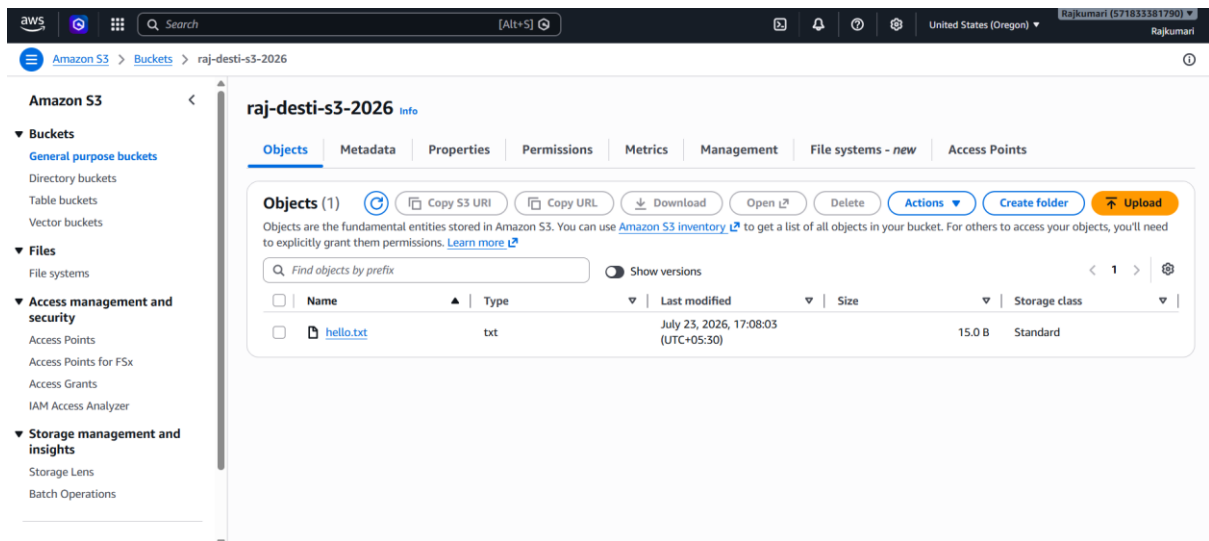
Step 10: Upload a New Object

1. Open the **source bucket**.
2. Click **Upload**.
3. Upload a file, for example:
 - sample.txt
4. Wait for the upload to complete.

Step 11: Verify Replication

1. Open the **destination bucket**.
2. Refresh the page after a short wait (replication is usually completed within a few minutes).

3. Confirm that the uploaded object appears in the destination bucket.



4. Configure a bucket policy so only the Admin user can see the objects of the S3 bucket.

Objective: Restrict access to an S3 bucket so that **only the IAM user named Admin** can list the bucket contents and view/download the objects.

Assumption: Your IAM user is named **Admin**. If your username is different, replace Admin with the correct username.

Step 1: Verify the IAM User

1. Sign in to the AWS Management Console.
2. Open IAM.
3. Click **Users**.

Here, Admin = rajkumari

4. Confirm that a user named **rajkumari** exists.

Step 2: Copy the rajkumari User ARN

1. Click the **rajkumari** user.
2. Copy the **ARN**.

Example:

arn:aws:iam::123456789012:user/rajkumari

Step 3: Open the S3 Bucket

1. Go to **Amazon S3**.
2. Click your bucket.

Example:

rajkumari-s3-2026

Step 4: Open Bucket Policy

1. Select the **Permissions** tab.
2. Scroll to **Bucket policy**.
3. Click **Edit**.

Step 5: Add the Bucket Policy

Replace:

- YOUR-BUCKET-NAME
- 123456789012
- Admin

The image shows two screenshots from the AWS Management Console. The top screenshot displays the 'Edit bucket policy' page for the bucket 'rajkumari-s3-2026'. The policy is a JSON document with two statements: 'AllowOnlyAdmin' and 'AllowOnlyAdminObjects'. The bottom screenshot shows the 'IAM user' page for 'rajkumari'. The 'Summary' section shows the user's ARN as 'arn:aws:iam::571833381790:user/rajkumari', console access as 'Enabled without MFA', and two access keys. The 'Permissions' section shows five policies attached to the user: 'AdministratorAccess' and 'AmazonEC2FullAccess' (both AWS managed and attached directly), and three custom policies (attached via groups).

Amazon S3 > Buckets > rajkumari-s3-2026 > Edit bucket policy

Amazon S3

- ▼ Buckets
 - General purpose buckets
 - Directory buckets
 - Table buckets
 - Vector buckets
- ▼ Files
 - File systems
- ▼ Access management and security
 - Access Points
 - Access Points for FSx
 - Access Grants
 - IAM Access Analyzer
- ▼ Storage management and insights
 - Storage Lens
 - Batch Operations

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "AllowOnlyAdmin",
6       "Effect": "Allow",
7       "Principal": {
8         "AWS": "arn:aws:iam::571833381790:user/rajkumari"
9       },
10      "Action": [
11        "s3:ListBucket"
12      ],
13      "Resource": "arn:aws:s3:::rajkumari-s3-2026/*"
14    },
15    {
16      "Sid": "AllowOnlyAdminObjects",
17      "Effect": "Allow",
18      "Principal": {
19        "AWS": "arn:aws:iam::571833381790:user/rajkumari"
20      },
21      "Action": [
22        "s3:GetObject"
23      ],
24      "Resource": "arn:aws:s3:::rajkumari-s3-2026/*"
25    }
26  ]
27 }
```

rajkumari info

Summary

ARN: [arn:aws:iam::571833381790:user/rajkumari](#)

Console access: [Enabled without MFA](#)

Access key 1: AKIA... - Active (Used Yesterday, 8 days old)

Access key 2: [Create access key](#)

Created: July 14, 2026, 23:44 (UTC+05:30)

Last console sign-in: [Never](#)

Permissions | Groups | Tags (1) | Security credentials | Last Accessed

Permissions policies (5)

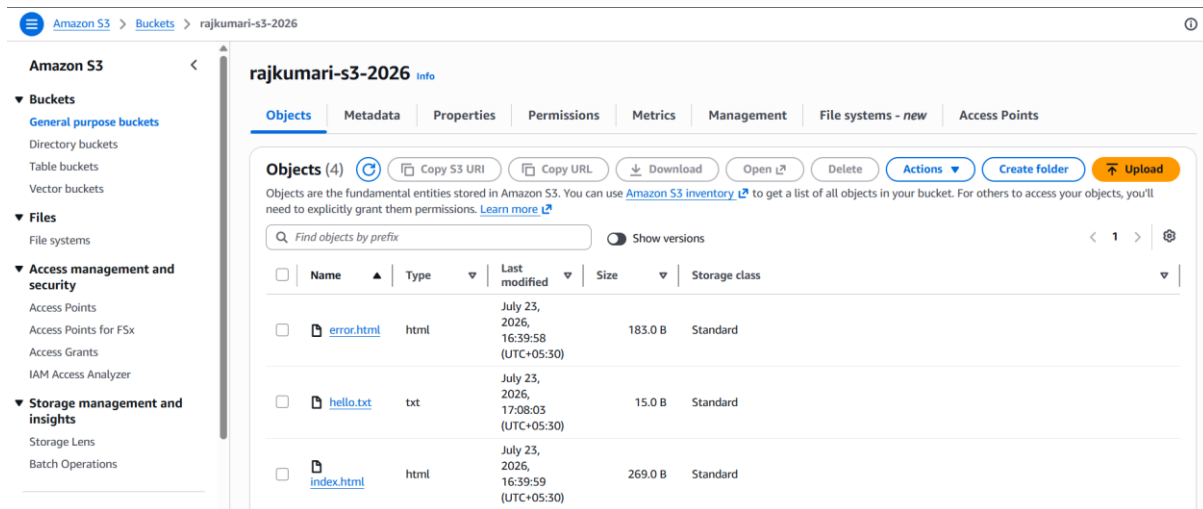
Permissions are defined by policies attached to the user directly or through groups.

Policy name	Type	Attached via
AdministratorAccess	AWS managed - job function	Directly
AmazonEC2FullAccess	AWS managed	Directly

How to verify

Test 1: Login as rajkumari

- Open S3.
- Open bucket rajkumari-s3-2026.
- You should be able to:
 - ☒ View the bucket
 - ☒ List objects
 - ☒ Download objects



5. Set up lifecycle policies to automatically transition or delete objects based on specific criteria.

Objective: Create an **S3 Lifecycle Rule** that automatically transitions objects to a different storage class or deletes them after a specified number of days.

What is an S3 Lifecycle Policy?

An S3 Lifecycle Policy is a set of rules that automatically manages objects over time, helping reduce storage costs and clean up old data.

Prerequisites

- ☒ AWS account
- ☒ An existing S3 bucket (for example, rajkumari-s3-2026)
- ☒ At least one object uploaded to the bucket

Step 1: Open Amazon S3

1. Sign in to the **AWS Management Console**.
2. Search for **S3**.
3. Open **Amazon S3**.

4. Click your bucket.

Example:

rajkumari-s3-2026

Step 2: Open the Management Tab

1. Inside the bucket, click the **Management** tab.
2. Scroll to **Lifecycle rules**.
3. Click **Create lifecycle rule**.

Step 3: Enter Rule Details

Lifecycle rule name

Example:

DeleteOldObjects

Rule scope

Select:

Apply to all objects in the bucket

Check the acknowledgment box confirming that the rule applies to all objects.

Click **Next**

Step 4: Choose Lifecycle Actions

You can configure one or both of the following actions.

Option A: Transition Objects to a Lower-Cost Storage Class

1. Select **Transition current versions of objects between storage classes**.
2. Configure, for example:
 - **Transition after:** 30 days

- **Storage class:** S3 Standard-IA

This moves objects to a lower-cost storage class after 30 days.

Step 6: Verify the Lifecycle Rule

Go back to:

Bucket → Management → Lifecycle rules

You should see:

Rule Name	Status
-----------	--------

DeleteOldObjects	Enabled
------------------	---------

objects. Lifecycle rules run once per day.

Default minimum object size for transitions
All storage classes 128K

Lifecycle rules (1)

[View details](#) [Edit](#) [Delete](#) [Actions](#) [Create lifecycle rule](#)

Use lifecycle rules to define actions you want Amazon S3 to take during an object's lifetime such as transitioning objects to another storage class, archiving them, or deleting them after a specified period of time. [Learn more](#)

	Lifecycl...	Status	Scope	Current ...	Noncurr...	Expirec
☐	DeleteOldObjects	Enabled	Entire bucket	Transition to Sta	-	-

6. Push some objects to S3 using the AWS CLI.

Objective: Upload one or more files (objects) from your local machine to an Amazon S3 bucket using the AWS Command Line Interface (CLI).

Prerequisites

Before starting, ensure:

- ☒ AWS CLI is installed.
- ☒ AWS CLI is configured with valid IAM credentials.
- ☒ An S3 bucket already exists (for example, rajkumari-s3-2026).

Step 1: Verify AWS CLI Installation

Open **Command Prompt**, **PowerShell**, or **Git Bash** and run: `$ aws --version`

Step 2: Configure AWS CLI

If you haven't configured it yet, run: `$ aws configure`

Step 3: Verify the Bucket

List all buckets: `$ aws s3 ls`

```
C:\Users\brajk>aws --version
aws-cli/2.36.6 Python/3.14.6 Windows/11 exe/AMD64

C:\Users\brajk>aws configure
AWS Access Key ID [*****27GJ]: AKIAYKI7M60PPZD327GJ
AWS Secret Access Key [*****lpcj]: TRwB0Jvw7YiRnQ2eaLXIR/P4
Default region name [us-east-1]: us-east-1
Default output format [json]: json

C:\Users\brajk>aws s3 ls
2026-07-23 16:56:18 raj-desti-s3-2026
2026-07-23 16:17:11 rajkumari-s3-2026
2026-07-16 00:05:16 vpc-flowlogs-bucket-raji

C:\Users\brajk>
```

Step 4: Create a Sample File

Create a simple text file.

Windows (Command Prompt): `$ echo Hello AWS S3 > sample.txt`

Step 5: Upload the File

Use the following command: `$ aws s3 cp demo.txt s3://rajkumari-s3-2026/`

Step 6: Verify the Upload

List the objects in the bucket: `$ aws s3 ls s3://rajkumari-s3-2026/`

```
C:\Users\brajk>aws --version
aws-cli/2.36.6 Python/3.14.6 Windows/11 exe/AMD64

C:\Users\brajk>aws configure
AWS Access Key ID [*****27GJ]: AKIAYKI7M60PPZD327GJ
AWS Secret Access Key [*****lpcj]: TRwB0Jvw7YiRnQ2eaLXIR/P4zouRBDGHTNWglpcj
Default region name [us-east-1]: us-east-1
Default output format [json]: json

C:\Users\brajk>aws s3 ls
2026-07-23 16:56:18 raj-desti-s3-2026
2026-07-23 16:17:11 rajkumari-s3-2026
2026-07-16 00:05:16 vpc-flowlogs-bucket-raji

C:\Users\brajk>echo "Hello AWS S3" > demo.txt

C:\Users\brajk>aws s3 cp demo.txt s3://rajkumari-s3-2026/
upload: .\demo.txt to s3://rajkumari-s3-2026/demo.txt

C:\Users\brajk>aws s3 ls s3://rajkumari-s3-2026/
2026-07-23 19:02:04      17 demo.txt
2026-07-23 16:39:58    183 error.html
2026-07-23 17:08:03     15 hello.txt
2026-07-23 16:39:59    269 index.html
2026-07-23 16:23:34     15 sample.txt

C:\Users\brajk>
```

Step 7: Upload Multiple Files

Suppose you have:

resume.pdf

image.png

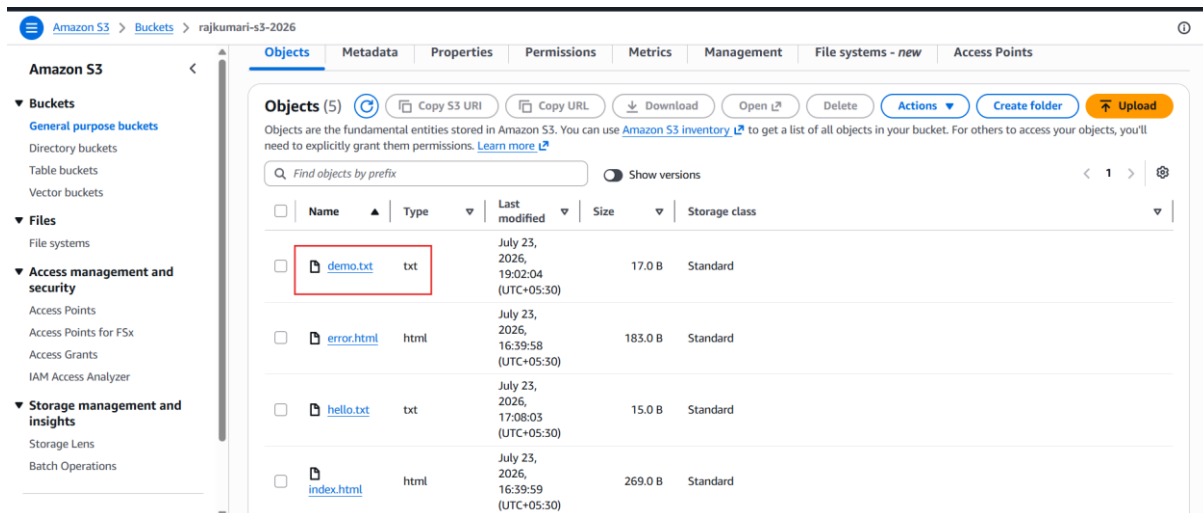
notes.txt

Upload all files: `$ aws s3 cp . s3://rajkumari-s3-2026/ --recursive`

This uploads all files in the current directory and its subdirectories.

Step 8: Verify from AWS Console

1. Open the **Amazon S3** console.
2. Open the bucket `rajkumari-s3-2026`.
3. Confirm that `sample.txt` (and any other uploaded files) are visible.



7. Write a Bash script to create an S3 bucket.

Objective: Create an Amazon S3 bucket automatically using a Bash script and AWS CLI commands.

Step 1: Verify AWS CLI Installation

First check whether AWS CLI is installed. Run : `$ aws --version`

Step 2: Configure AWS CLI

Configure AWS credentials so the script can communicate with AWS.

Run: `$ aws configure`

Step 3: Create Bash Script File

Create a script file: `$ vi create-s3-bucket.sh`

Step 4: Give Execute Permission

By default, scripts do not have execute permission.

Run: `$ chmod +x create-s3-bucket.sh`

Run: `$ ls -l create-s3-bucket.sh`

Step 5: Execute the Script

Run: `$ ./create-s3-bucket.sh`

Step 6: Verify S3 Bucket Creation

List all S3 buckets: `$ aws s3 ls`

```
brajk@RajKumari MINGW64 ~/Downloads
$ $ vi create-s3-bucket.sh
bash: $: command not found

brajk@RajKumari MINGW64 ~/Downloads
$ vi create-s3-bucket.sh

brajk@RajKumari MINGW64 ~/Downloads
$ chmod +x create-s3-bucket.sh

brajk@RajKumari MINGW64 ~/Downloads
$ ls -l create-s3-bucket.sh
-rwxr-xr-x 1 brajk 197609 377 Jul 23 19:15 create-s3-bucket.sh*

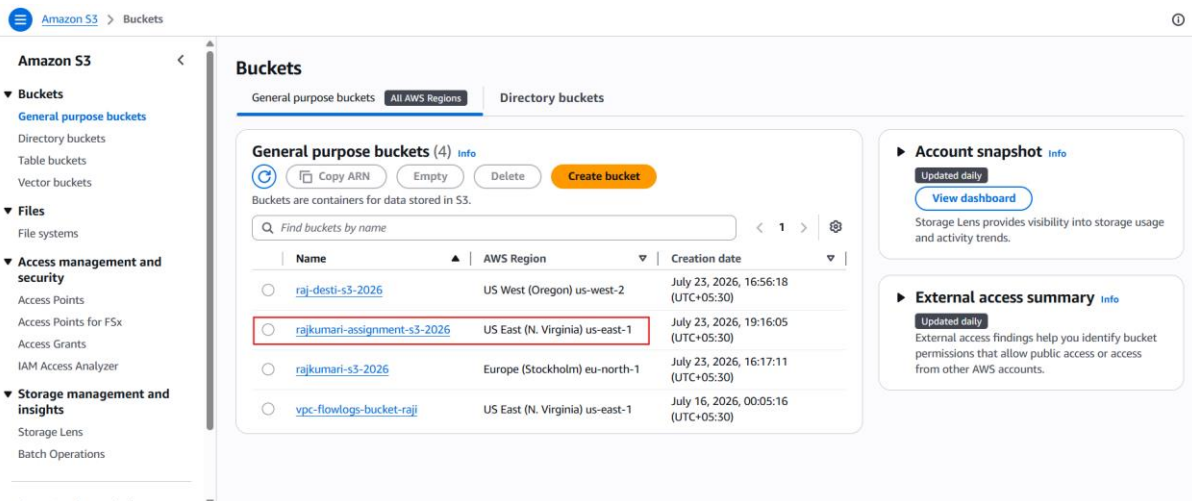
brajk@RajKumari MINGW64 ~/Downloads
$ ./create-s3-bucket.sh
Creating S3 Bucket: rajkumari-assignment-s3-2026
{
  "Location": "/rajkumari-assignment-s3-2026",
  "BucketArn": "arn:aws:s3:::rajkumari-assignment-s3-2026"
}
S3 Bucket created successfully

brajk@RajKumari MINGW64 ~/Downloads
$ aws s3 ls
2026-07-23 16:56:18 raj-desti-s3-2026
2026-07-23 19:16:05 rajkumari-assignment-s3-2026
2026-07-23 16:17:11 rajkumari-s3-2026
2026-07-16 00:05:16 vpc-flowlogs-bucket-raji

brajk@RajKumari MINGW64 ~/Downloads
$ |
```

You can also verify from AWS Console:

1. Open AWS Console
2. Go to S3
3. Click **Buckets**
4. Verify your bucket name



```
C:\Users\brajk>aws sts get-caller-identity
{
  "UserId": "AIDAYKI7M60PFEHIVCGC3",
  "Account": "571833381790",
  "Arn": "arn:aws:iam::571833381790:user/rajkumari"
}

C:\Users\brajk>aws s3 ls
2026-07-23 16:56:18 raj-desti-s3-2026
2026-07-23 19:16:05 rajkumari-assignment-s3-2026
2026-07-23 16:17:11 rajkumari-s3-2026
2026-07-16 00:05:16 vpc-flowlogs-bucket-raji

C:\Users\brajk>
```

8. Upload a 1 GB file to S3 using the CLI.

Objective: Create a 1 GB file and upload it to an Amazon S3 bucket using the AWS CLI.

Prerequisites

- AWS CLI installed
- AWS CLI configured (aws configure)
- An existing S3 bucket
- IAM user with:
 - s3: PutObject
 - s3: ListBucket

Step 1: Verify AWS CLI: \$ aws --version

Step 2: Verify AWS Credentials: \$ aws sts get-caller-identity

Step 3: Verify Your S3 Bucket: \$ aws s3 ls

List all buckets:

```
[ec2-user@ip-172-31-43-40 ~]$ aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.18.38-73.137.amzn2023.x86_64 source/x86_64
.amzn.2023
[ec2-user@ip-172-31-43-40 ~]$ aws configure
AWS Access Key ID [None]: AKIAWKI7M60PPZD327GJ
AWS Secret Access Key [None]: TRwB0Jvw7YiRnQ2eaLXIR/P4zouRBDGHTNWglpcj
Default region name [None]: us-east-1
Default output format [None]: json
[ec2-user@ip-172-31-43-40 ~]$ aws sts get-caller-identity
{
  "UserId": "AIDAYKI7M60PFEHIVCGC3",
  "Account": "571833381790",
  "Arn": "arn:aws:iam::571833381790:user/rajkumari"
}
[ec2-user@ip-172-31-43-40 ~]$ aws s3 ls
2026-07-23 11:26:18 raj-desti-s3-2026
2026-07-23 13:46:05 rajkumari-assignment-s3-2026
2026-07-23 10:47:11 rajkumari-s3-2026
2026-07-15 18:35:16 vpc-flowlogs-bucket-raj
[ec2-user@ip-172-31-43-40 ~]$ |
```

Step 4: Create a 1 GB File

On Linux (EC2/Amazon Linux): \$ fallocation -l 1G samplefile.bin

If fallocation is unavailable: \$ dd if=/dev/zero of=samplefile.bin bs=1M
count=1024

\$ ls -lh samplefile.bin

Step 5: Upload the File to S3

Replace the bucket name with your bucket.

\$ aws s3 cp samplefile.bin s3://rajkumari-assignment-s3-2026/

Step 6: Verify the Upload

List the bucket contents: \$ aws s3 ls s3://rajkumari-assignment-s3-2026/

```
[ec2-user@ip-172-31-43-40 ~]$ fallocation -l 1G samplefile.bin
[ec2-user@ip-172-31-43-40 ~]$ ls -lh samplefile.bin
-rw-r--r--. 1 ec2-user ec2-user 1.0G Jul 23 14:05 samplefile.bin
[ec2-user@ip-172-31-43-40 ~]$
aws s3 cp samplefile.bin s3://rajkumari-assignment-s3-2026/
upload: ./samplefile.bin to s3://rajkumari-assignment-s3-2026/samplefile.bin
[ec2-user@ip-172-31-43-40 ~]$ aws s3 ls s3://rajkumari-assignment-s3-2026/
2026-07-23 14:07:18 1073741824 samplefile.bin
[ec2-user@ip-172-31-43-40 ~]$ |
```

Step 7: Verify in AWS Console

1. Sign in to the AWS Management Console.
2. Open **Amazon S3**.
3. Select your bucket.
4. Confirm that samplefile.bin appears in the object list with a size of approximately **1 GB**.

The screenshot shows the AWS Management Console interface for the Amazon S3 bucket named 'rajkumari-assignment-s3-2026'. The left sidebar displays the navigation menu with categories like Buckets, Files, Access management and security, and Storage management and insights. The main content area shows the 'Objects' tab with a table listing the objects in the bucket. A single object, 'samplefile.bin', is listed with a size of 1.0 GB and a storage class of Standard. The object's name, size, and type are highlighted with red boxes in the original image.

Name	Type	Last modified	Size	Storage class
samplefile.bin	bin	July 23, 2026, 19:37:18 (UTC+05:30)	1.0 GB	Standard